claude-windows / 144a31b7-7962-4fee-9d6f-8a7854daa63f

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\144a31b7-7962-4fee-9d6f-8a7854daa63f\subagents\workflows\wf_887fb14e-e6d\agent-ad9b2b1329f26e05e.jsonl`
- SHA-256: `30db3ed98ce55f230f4cc868d90d372e0b21e2dc2f1cb715b131b08db29f008c`
- Source modified: `2026-06-20T10:05:39+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_887fb14e-e6d`
- session_id: `144a31b7-7962-4fee-9d6f-8a7854daa63f`

Transcript

1. user (2026-06-20T10:03:00.484Z)

Goal: design dedicated Studio-UI pages that turn "annotate your footage ? train your OWN model" into a first-class USP (the bring-your-own-model moat), linking the public GitHub repo (https://github.com/avidullu/rally-annotator), with screenshots and step-by-step setup/use help. Be HONEST about what is real today vs roadmap. Tag claims [verified]/[design]/[researched]. Do NOT invent capabilities. EXCLUDE any .claude/worktrees/ paths.

In the engine repo (cwd=C:\\Users\\avidu\\Projects\\badminton-highlight-indexer) map the "TRAIN YOUR OWN MODEL" loop that consumes hand-labels. Read backend/worker/__main__.py (the `train` command), the label CSV schema it expects (labels/*.csv), docs/SETUP_NEW_MACHINE.md, docs/DECOUPLED_COMPUTE/* (train-from-bucket), and training/ (gen0 harness, shelled out). Report: the annotate?CSV?`worker train`?weights flow; the EXACT engine label-CSV schema; whether the rally-annotator output CSV MATCHES that schema (precise gaps/transforms); the concrete train commands; and the GATING (real train needs a GPU + native WASB [M3.5, in-flight] for real trajectories, else synth/stub mock; Gemini is inference-only, never a training source per the licensing guardrail). Quote path:line. Return the schema.

2. assistant (2026-06-20T10:03:03.270Z)

I'll map the "TRAIN YOUR OWN MODEL" loop in the engine repo. Let me start by pulling the repo fresh and reading the key files.

3. user (2026-06-20T10:03:06.537Z)

Already up to date.

4. user (2026-06-20T10:03:06.543Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
```

```
    (validator registry ? segmenter ? report), never a forked pipeline.
    """
    from __future__ import annotations

    import argparse
    import csv
    import json
    import os
    import sys
    import tempfile
    from typing import Any, List

    from backend import storage
    from backend.config import load_config
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.validators.base import registry as validator_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure
    from backend.utils.hashing import compute_video_id


    def _coerce(v: str) -> Any:
        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
        low = v.lower()
        if low in ("true", "false"):
            return low == "true"
        for cast in (int, float):
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow([
                    f'{float(s.get("start_time", 0.0)):.3f}',
                    f'{float(s.get("end_time", 0.0)):.3f}',
                    s.get("shot_count", ""),
                    s.get("ending_reason", s.get("shot_count_confidence", "")),
                ])

```

```
76
77      def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
78          with open(path, "w") as f:
79              json.dump({
80                  "video_id": video_id,
81                  "status": status,
82                  "segment_count": len(segments),
83                  "segments": [{"start": float(s.get("start_time", 0.0)),
84                                "end": float(s.get("end_time", 0.0))} for s in segments],
85              }, f, indent=2)
86
87
88      def cmd_infer(args: argparse.Namespace) -> int:
89          config = load_config(args.config) if args.config else load_config()
90          _apply_overrides(config, args.set)
91          storage_cfg = config.storage.model_dump()
92
93          scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
94          os.makedirs(scratch, exist_ok=True)
95          config.output.output_dir = scratch
96
97          # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
98          local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
99          print(f"[worker] pulled {args.video_uri} -> {local_video}")
100
101         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
102         video_id = compute_video_id(local_video)
103
104         # 3. VALIDATE (same registry as the main CLI).
105         val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
106         failures = [r for r in val_results if not r.passed]
107         db = Database(os.path.join(scratch, config.output.db_name))
108         db.add_video(video_id, local_video, "failed" if failures else "processed",
109                      [r.model_dump() for r in val_results])
110
111         segments: List[dict] = []
112         segmenter_actual = None
113         segmentation_ran = False
114         status = "failed"
115         try:
116             if failures:
117                 print("[worker] validation FAILED: "
118                       + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
119                 return 2
120             seg_name = args.segmenter or config.indexing.default_segmenter
121             segmenter_cls = segmenter_registry.get(seg_name)
122             if not segmenter_cls:
123                 print(f"[worker] unknown segmenter: {seg_name!r} "
124                       f"(have {list(segmenter_registry.list_available())})")
125                 return 2
126             segmenter_actual = seg_name
127             result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
128             segmentation_ran = True
129             if isinstance(result, Failure):
130                 print(f"[worker] segmenter FAILED: {result.message}")
131                 return 3
132             segments = result.value if hasattr(result, "value") else result
133             status = "success"
134         finally:
```

```
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db, video_id=video_id, video_path=local_video, config=config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=False, segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))

        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(out_local)):
            uri = f"{out_prefix}/outputs/{video_id}/{fn}"
            storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
            pushed.append(uri)

        print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
    #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
    _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")


    def _probe_video_fps_width(path: str):
        """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.

        The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
        fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
        cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
        inference report (it flags the fallback) but corrupting for REAL training data. So
here we
        probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
        rather than guesses.
        """
        import json
        import subprocess
        try:
            out = subprocess.check_output(
                ["ffprobe", "-v", "error", "-select_streams", "v:0",
                 "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
                stderr=subprocess.DEVNULL).decode()
            st = (json.loads(out).get("streams") or [{}])[0]
            num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
            fps = float(num) / float(den or "1")
            width = float(st.get("width") or 0)
            if fps > 0 and width > 0:
                return fps, width
```

```
        except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
            pass
        return None


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(f"[worker] unknown segmenter: {args.segmenter!r} "
                  f"(have {list(segmenter_registry.list_available())})")
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                  f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
            return None
        try:
            runner, _ = seg._resolve_runner(config.get("indexing", {}))
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
            shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
          * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
            DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
            is the M3.5 "first real training" path; only the trajectory source flips.
        """
        import subprocess

```

```
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
        labels_dir = os.path.join(scratch, "labels")
        traj_dir = os.path.join(scratch, "trajectories")
        videos_dir = os.path.join(scratch, "videos")
        os.makedirs(labels_dir, exist_ok=True)
        os.makedirs(traj_dir, exist_ok=True)

        corpus = args.corpus_uri.rstrip("/")
        label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
                            if u.lower().endswith(".csv"))
        if len(label_uris) < 2:
            print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                  f"under {corpus}/labels/")
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                if not vname.lower().endswith(_VIDEO_EXTS):
                    continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
            name = fname.replace(".rallies.csv", "").replace(".csv", "")
            local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
                    continue
                local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
                                            config=storage_cfg)
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
                    continue
                fps, frame_width = meta
```

```
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
                    continue
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": fps, "frame_width": frame_width})
            else:
                from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": args.fps, "frame_width": args.frame_width})

        if len(specs) < 2:
            print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [sys.executable, "-m", "training.gen0.harness",
               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
        if args.floor:
            floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
                           if "://" in args.floor else args.floor)
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(prog="python -m backend.worker",
                                    description="Storage-aware worker: pull ? run pipeline ?
push.")
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
        pi.add_argument("--out-uri", required=True, help="output prefix
```

```
(outputs/<video_id>/? is appended)")
355         pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
356         pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
357         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
358         pi.add_argument("--max-frames", type=int, default=None)
359         pi.add_argument("--set", action="append", default=[], metavar="k=v",
360                         help="config override, e.g. --set indexing.skip_ai_handoff=true")
361         pi.set_defaults(func=cmd_infer)
362
363         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
364         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
365                         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
366         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
367         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
368         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
369                         help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
370                              "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
371                              "(needs videos/ in the corpus; honours detector_impl).")
372         pt.add_argument("--segmenter", default="wasb_hybrid",
373                         help="trajectory hybrid whose detector is used for --trajectory-
source detector")
374         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
375         pt.add_argument("--config", default=None)
376         pt.add_argument("--scratch", default=None)
377         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
378         pt.add_argument("--frame-width", type=float, default=1280.0,
379                         help="synth trajectory frame width (detector probes the video)")
380         pt.add_argument("--set", action="append", default=[], metavar="k=v")
381         pt.set_defaults(func=cmd_train)
382         return p
383
384
385     def main(argv=None) -> int:
386         args = build_parser().parse_args(argv)
387         return args.func(args)
388
389
390     if __name__ == "__main__":
391         sys.exit(main())
392
```

5. assistant (2026-06-20T10:03:10.815Z)

Now let me read the label CSV schema files, the rally-annotator output, and the key docs. Let me explore in parallel.

6. user (2026-06-20T10:03:13.505Z)

```
ls: cannot access 'C:/Users/avidu/Projects/badminton-highlight-indexer/labels/': No such file or
directory
---DOCS DECOUPLED---
total 124
drwxr-xr-x 1 avidu 197609     0 Jun 20 15:14 .
drwxr-xr-x 1 avidu 197609     0 Jun 20 15:25 ..
```

```
-rw-r--r-- 1 avidu 197609 12750 Jun 20 13:48 01-EXISTING-PLANS-AND-GAP.md
-rw-r--r-- 1 avidu 197609  6617 Jun 20 13:48 02-COMPUTE-MAP-AND-CONTRACT.md
-rw-r--r-- 1 avidu 197609  8293 Jun 20 13:48 03-PLATFORM-OPTIONS.md
-rw-r--r-- 1 avidu 197609 10551 Jun 20 13:48 04-BUILD-AND-TEST-PLAN.md
-rw-r--r-- 1 avidu 197609  6792 Jun 20 13:48 05-COST-ATTRIBUTION.md
-rw-r--r-- 1 avidu 197609  7608 Jun 20 13:48 06-RISKS-AND-OPEN-QUESTIONS.md
-rw-r--r-- 1 avidu 197609 11247 Jun 20 13:48 07-COMPUTE-ABSTRACTION.md
-rw-r--r-- 1 avidu 197609  7385 Jun 20 15:14 HANDOFF.md
-rw-r--r-- 1 avidu 197609  8773 Jun 20 13:48 README.md
---TRAINING---
C:/Users/avidu/Projects/badminton-highlight-indexer/training/gen0/harness.py
C:/Users/avidu/Projects/badminton-highlight-indexer/training/gen0/__init__.py
C:/Users/avidu/Projects/badminton-highlight-indexer/training/README.md
C:/Users/avidu/Projects/badminton-highlight-indexer/training/requirements-train.txt
C:/Users/avidu/Projects/badminton-highlight-indexer/training/__init__.py
```

7. user (2026-06-20T10:03:15.959Z)

```
Already up to date.
---FILES---
./.gitattributes
./.github/workflows/ci.yml
./.gitignore
./CHANGELOG.md
./docs/CSV_FORMAT.md
./docs/ENDING_REASONS.md
./LICENSE
./README.md
./test/dialog_layout.snapshot
./test/dialog_test.lua
./test/README.md
./vlc/rally_annotator.lua
```

8. assistant (2026-06-20T10:03:28.152Z)

```
The labels/ dir doesn't exist at repo root. Let me find where labels live and read the key
files.
```

9. user (2026-06-20T10:03:31.620Z)

```
1       # Output CSV format
2
3       One row per rally. Header is written once when the file is created.
4
5       ```
6       rally_number,start_time,end_time,ending_reason,sport,shots_count
7       1,8.800,11.500,winner,badminton,9
8       2,24.389,46.589,unforced_error,badminton,21
9       3,49.183,54.683,let,badminton,
10      ```
11
12      | Column | Type | Notes |
13      |---|---|---|
14      | `rally_number` | int | 1-based, monotonic; continues across re-opening the same CSV |
15      | `start_time` | float | rally start, **decimal seconds** from the video start |
16      | `end_time` | float | rally end, decimal seconds (always `> start_time`) |
17      | `ending_reason` | enum | `unknown` (default) / `winner` / `forced_error` /
`unforced_error` / `service_fault` / `let` / `other` ? see the decision guide in
[ENDING_REASONS.md](ENDING_REASONS.md) |
18      | `sport` | enum | `badminton` / `tennis` / `table_tennis` / `pickleball` / `padel` |
19      | `shots_count` | int | **optional** ? number of shots/strokes in the rally. **Blank**
when not entered (as in row 3 above); added in v1.6. Older CSVs without this column still load.
|
20
```

```
21      Notes:
22      - **Decimal seconds**, not `mm:ss` ? downstream loaders parse floats.
23      - Rows are appended live; **Undo last** truncates the last row (one level).
24      - Consumers should read by **column name** (the header is stable but column *order* may
evolve; extra
25        columns may be added later). Most CSV `DictReader`-style loaders already tolerate
extra columns.
26
27      ## Ingesting the labels
28
29      This CSV is intentionally compatible with common rally-segmentation tooling that reads
30      `rally_number,start_time,end_time` by header name and drops rows where `end <= start`:
31
32      ```bash
33      # Generic: any tool that takes a labels CSV of start/end seconds.
34
35      # Example ? score a detector against these labels, or slice clips by them
36      # (badminton-highlight-indexer loaders read --labels <this.csv> directly):
37      python -m backend.eval.calibrate_wasb --trajectory traj.csv --labels match.rallies.csv
--fps 59.94 --frame-width 1920
38      python -m backend.tools.rally_slicer --video match.mp4 --labels match.rallies.csv
--rallies all
39
40      # Example ? register as curated golden data in sports-data-collector:
41      python -m src.cli.curate import-local --sport badminton --video-path match.mp4
--annotations-path match.rallies.csv
42      ```
43
44      The `sport` column is self-documenting; tools that take a `--sport` flag can ignore it.
45
```

10. user (2026-06-20T10:03:32.216Z)

```
1       """Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->
artifact.
2
3           python -m training.gen0.harness --manifest output/human_lovo_manifest.json \
4               --floor eval_baselines/heuristic_lovo_n6.json --out artifacts/rally_tal_gen0
--version 0.1.0
5
6       Productionizes the rally_seq prototype per ADR-008:
7       - TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
8         honest leave-one-video-out (split by match, threshold tuned on train only), then fits
9         the FINAL model on all videos.
10      - Emits a versioned ARTIFACT BUNDLE ? the only thing serving may ever consume:
11            <out>/<version>/weights.json    (gitignored; classifier + frozen normalization)
12            <out>/<version>/manifest.json   (the reviewable contract: feature-schema version,
13                                             calibration, training lineage + GT hashes,
14                                             eval results, attestations, gate verdict)
15      - Runs the SHIP-GATE as pre-flight: floor comparison + a PAIRED per-video sign test
16        (at n=6 a mean gap alone is noise-ambiguous), plus the hard attestation blocks
17        (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
18        undefined => no self-declared victory). The product side owns final admission
19        (backend/eval/regression.py); this harness never grades itself into serving.
20
21      Training may import backend.* (one-way wall); backend must never import training.*.
22      """
23      from __future__ import annotations
24
25      import argparse
26      import hashlib
27      import json
28      import os
29      import subprocess
30      import time
```

```
    from math import comb
    from typing import Dict, List, Optional

    import numpy as np

    from backend.eval.classifier import LogisticRegression
    from backend.eval.gt_loader import load_gt_intervals
    from backend.eval.regression import gt_hash
    from backend.eval.rally_seq_proto import (
        labels_for, run as lovo_run, _seg_f1,
    )
    from backend.features.rally_seq import FEAT_NAMES, FEATURE_SCHEMA, build_frame_arrays,
features
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    # Post-processing constants ? part of the model's identity, recorded in the manifest.
    CALIBRATION_DEFAULTS = {"smooth": 7, "merge_gap": 1.0, "min_dur": 1.0}
    THRESHOLD_GRID = [round(t, 2) for t in np.arange(0.2, 0.81, 0.05)]


    def _git_sha() -> str:
        try:
            return subprocess.run(["git", "rev-parse", "HEAD"], capture_output=True,
                                  text=True, timeout=10).stdout.strip() or "unknown"
        except Exception:
            return "unknown"


    def _sha256(path: str) -> str:
        h = hashlib.sha256()
        with open(path, "rb") as f:
            for chunk in iter(lambda: f.read(65536), b""):
                h.update(chunk)
        return h.hexdigest()


    def sign_test(learned: List[float], floor: List[float]):
        """Paired one-sided sign test: is learned > floor more often than chance?

        At n=6 a mean comparison alone is inside noise (SE ~0.11); the paired per-video
        wins are the defensible statistic. Returns (wins, n, p_one_sided)."""
        pairs = [(lv, fv) for lv, fv in zip(learned, floor) if lv != fv]  # ties drop,
standard
        wins = sum(1 for lv, fv in pairs if lv > fv)
        n = len(pairs)
        p = sum(comb(n, k) for k in range(wins, n + 1)) / (2 ** n) if n else 1.0
        return wins, n, p


    def train_final(specs: List[Dict]):
        """Fit the FINAL model on ALL corpus videos; threshold tuned on the same (train-
side).

        The honest generalization estimate is the LOVO result, not this fit ? recorded as
such."""
        vids = []
        for s in specs:
            pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
            arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
            X, times = features(arr, float(s["fps"]))
            gts = load_gt_intervals(s["labels"], strict=False)
            vids.append({"name": s["name"], "X": X, "times": times,
                         "y": labels_for(times, gts), "gts": gts})
        Xall = np.vstack([v["X"] for v in vids])
        yall = np.concatenate([v["y"] for v in vids])
```

```
92          clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
93          clf.fit(Xall, yall, feature_names=FEAT_NAMES)
94          thr = max(THRESHOLD_GRID, key=lambda t: _seg_f1(clf, vids, float(t)))
95          return clf, float(thr)
96
97
98      def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
99          """Pre-flight ship-gate.
100
101         Provisional F1 target (NOT owner-ratified; calibration tracked in #172).
102         Floor (necessary): heuristic LOVO ~0.480 (from quality table).
103         Multi-court ref (makes owned \"worth serving\" per data): 0.66.
104         Proposed for \"very high P/R\" + significance: F1@tIoU0.5 >= 0.70 on
105         multi-court folds with paired-sign p<0.05 vs heuristic (refine as corpus grows).
106         Beating floor + sign test necessary; C3 main commercial blocker.
107         ship=True only when all non-C3 blocks clear (honesty first).
108         """
109         reasons: List[str] = []
110         floor_passed = None
111         sign = None
112         if floor:
113             floor_mean = float(floor["mean_f1"])
114             floor_passed = eval_res["mean_f1"] > floor_mean
115             if not floor_passed:
116                 reasons.append(f"LOVO mean {eval_res['mean_f1']:.3f} <= floor
{floor_mean:.3f}")
117             per = floor.get("per_video", {})
118             paired = [(r["f1"], per[r["video"]]) for r in eval_res["per_video"] if
r["video"] in per]
119             if paired:
120                 wins, n, p = sign_test([lv for lv, _ in paired], [fv for _, fv in paired])
121                 sign = {"wins": wins, "n": n, "p_one_sided": round(p, 4)}
122                 if p >= 0.05:
123                     reasons.append(f"paired sign test not significant ({wins}/{n} wins,
p={p:.3f}) "
124                                    f"? grow the corpus before claiming superiority")
125         else:
126             reasons.append("no floor baseline provided ? nothing to beat")
127
128         # Provisional blocker (per owner review on #170; see #172 for calibration).
129         # (The number itself is not the blocker; calibration tracked in #172.)
130         reasons.append("ship-gate F1@tIoU target NOT YET CALIBRATED ? provisional; see issue
#172")
131         reasons.append("WASB checkpoint provenance C3 UNRESOLVED ? commercial ship blocked
regardless of F1")
132         # ship remains False until C3 resolved + other gates (by design).
133         return {"ship": False, "floor_passed": floor_passed, "sign_test": sign, "blockers":
reasons}
134
135
136     def run(manifest_path: str, out_dir: str, version: str,
137             floor_path: Optional[str] = None) -> dict:
138         with open(manifest_path, encoding="utf-8") as f:
139             specs = json.load(f)
140         floor = None
141         if floor_path:
142             with open(floor_path, encoding="utf-8") as f:
143                 floor = json.load(f)
144
145         # 1) Honest LOVO eval (the number that matters).
146         eval_res = lovo_run(specs)
147
148         # 2) Final model on all videos.
149         clf, thr = train_final(specs)
150
```

```
        # 3) Gate (pre-flight).
        verdict = gate_verdict(eval_res, floor)

        # 4) Artifact bundle.
        bundle_dir = os.path.join(out_dir, version)
        os.makedirs(bundle_dir, exist_ok=True)
        weights_path = os.path.join(bundle_dir, "weights.json")
        payload = clf.to_dict()
        payload["calibration"] = {"threshold": thr, **CALIBRATION_DEFAULTS}
        payload["feature_schema_version"] = FEATURE_SCHEMA["version"]
        with open(weights_path, "w", encoding="utf-8") as f:
            json.dump(payload, f, indent=2)

        manifest = {
            "artifact": {
                "name": "rally_tal_gen0", "version": version,
                "created_at": time.strftime("%Y-%m-%dT%H:%M:%S%z"),
                "git_sha": _git_sha(),
                "weights_file": "weights.json", "weights_sha256": _sha256(weights_path),
            },
            "feature_schema": FEATURE_SCHEMA,
            "calibration": {"threshold": thr, **CALIBRATION_DEFAULTS,
                            "note": "threshold tuned train-side on the full corpus; "
                                    "honest generalization = the LOVO eval below"},
            "training_lineage": {
                "corpus_manifest": manifest_path,
                "split_protocol": "leave-one-video-out by match",
                "label_source": "human (rally-annotator golden CSVs)",
                "videos": [{"name": s["name"], "labels": s["labels"],
                            "gt_hash": gt_hash(load_gt_intervals(s["labels"],
strict=False)),
                            "fps": s["fps"]} for s in specs],
            },
            "eval": {
                "protocol": "LOVO, F1@tIoU=0.5 segment-level + per-frame AUC",
                "per_video": eval_res["per_video"],
                "mean_f1": eval_res["mean_f1"],
                "floor": floor,
            },
            "attestation": {
                "deps_licenses": "numpy/scipy (BSD) + repo-internal code only (Gen-0)",
                "wasb_checkpoint_provenance": "C3-UNRESOLVED (ship-blocking; see
COMMERCIALIZATION C3)",
                "no_paid_llm_training_labels": True,
                "consent_ledger": "pending-M0.1 (owner-attested: owned footage, all-adult
sessions)",
            },
            "gate": verdict,
        }
        manifest_out = os.path.join(bundle_dir, "manifest.json")
        with open(manifest_out, "w", encoding="utf-8") as f:
            json.dump(manifest, f, indent=2)

        print(f"\n=== Gen-0 artifact bundle: {bundle_dir} ===")
        print(f"weights : {weights_path}")
        print(f"manifest: {manifest_out}")
        print(f"GATE    : ship={verdict['ship']}  floor_passed={verdict['floor_passed']}  "
              f"sign_test={verdict['sign_test']}")
        for b in verdict["blockers"]:
            print(f"  - {b}")
        return manifest


    def main() -> None:
        ap = argparse.ArgumentParser(description=__doc__)
```

```
213         ap.add_argument("--manifest", required=True,
214                         help="corpus manifest JSON:
[{name,trajectory,fps,frame_width,labels}, ...]")
215         ap.add_argument("--floor", default=None,
216                         help="floor baseline JSON (e.g.
eval_baselines/heuristic_lovo_n6.json)")
217         ap.add_argument("--out", default=os.path.join("artifacts", "rally_tal_gen0"))
218         ap.add_argument("--version", required=True, help="semver for the artifact bundle")
219         args = ap.parse_args()
220         run(args.manifest, args.out, args.version, floor_path=args.floor)
221
222
223     if __name__ == "__main__":
224         main()
225
```

11. user (2026-06-20T10:03:48.432Z)

```
===rally-annotator CSV docs===
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/eval_partial_labels.py
./.claude/worktrees/bhi-execpolicy/backend/eval/eval_partial_labels.py
./.claude/worktrees/boxhill17/backend/eval/eval_partial_labels.py
./.claude/worktrees/gap-doc/backend/eval/eval_partial_labels.py
./.claude/worktrees/hardchop/backend/eval/eval_partial_labels.py
./.claude/worktrees/pr-stitch/backend/eval/eval_partial_labels.py
./.claude/worktrees/recdoc/backend/eval/eval_partial_labels.py
./.claude/worktrees/resweep/backend/eval/eval_partial_labels.py
./.claude/worktrees/tier1-windowing/backend/eval/eval_partial_labels.py
./.claude/worktrees/w2-safeguard/backend/eval/eval_partial_labels.py
./backend/eval/eval_partial_labels.py
./backend/worker/__main__.py
./docs/DECOUPLED_COMPUTE/02-COMPUTE-MAP-AND-CONTRACT.md
./docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md
./docs/DECOUPLED_COMPUTE/HANDOFF.md
./docs/SETUP_NEW_MACHINE.md
./tests/test_worker.py
===find any sample label CSVs===
./.claude/worktrees/boxhill17/eval_baselines/fixtures/clean_single_rally/labels.csv
./.claude/worktrees/boxhill17/eval_baselines/fixtures/clean_single_rally/trajectory.csv
./.claude/worktrees/boxhill17/eval_baselines/fixtures/multi_rally_with_deadtime/labels.csv
./.claude/worktrees/boxhill17/eval_baselines/fixtures/multi_rally_with_deadtime/trajectory.csv
./.claude/worktrees/boxhill17/eval_baselines/fixtures/occlusion_break/labels.csv
./.claude/worktrees/boxhill17/eval_baselines/fixtures/occlusion_break/trajectory.csv
./.claude/worktrees/gap-doc/eval_baselines/fixtures/clean_single_rally/labels.csv
./.claude/worktrees/gap-doc/eval_baselines/fixtures/clean_single_rally/trajectory.csv
./.claude/worktrees/gap-doc/eval_baselines/fixtures/multi_rally_with_deadtime/labels.csv
./.claude/worktrees/gap-doc/eval_baselines/fixtures/multi_rally_with_deadtime/trajectory.csv
./.claude/worktrees/gap-doc/eval_baselines/fixtures/occlusion_break/labels.csv
./.claude/worktrees/gap-doc/eval_baselines/fixtures/occlusion_break/trajectory.csv
./.claude/worktrees/hardchop/eval_baselines/fixtures/clean_single_rally/labels.csv
./.claude/worktrees/hardchop/eval_baselines/fixtures/clean_single_rally/trajectory.csv
./.claude/worktrees/hardchop/eval_baselines/fixtures/multi_rally_with_deadtime/labels.csv
./.claude/worktrees/hardchop/eval_baselines/fixtures/multi_rally_with_deadtime/trajectory.csv
./.claude/worktrees/hardchop/eval_baselines/fixtures/occlusion_break/labels.csv
./.claude/worktrees/hardchop/eval_baselines/fixtures/occlusion_break/trajectory.csv
./.claude/worktrees/pr-stitch/eval_baselines/fixtures/clean_single_rally/labels.csv
./.claude/worktrees/pr-stitch/eval_baselines/fixtures/clean_single_rally/trajectory.csv
```

12. assistant (2026-06-20T10:03:51.714Z)

I have the rally-annotator schema and the harness. Now let me read the gt_loader (what the engine label CSV parser expects), the synth trajectory function, the SETUP doc, and the decoupled contract doc. Note: I must EXCLUDE .claude/worktrees/ paths.

13. user (2026-06-20T10:03:52.652Z)

```
1       """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3       The repo historically carried two incompatible loaders:
4       - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5         ``curate export`` format), raises on bad rows;
6       - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7         (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9       A file in one convention fed to the other loader yielded zero/garbage intervals with no
10      error. This module accepts BOTH conventions (header-mapped, else positional), so every
11      quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12      flows through one reader. Both legacy entry points now delegate here.
13
14      Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15      - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16        columns ? rally_number, ending_reason, sport, ? ? are ignored);
17      - no header: columns 0,1 positionally.
18      Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
(``annotations.parse_timestamp``).
19      """
20      import csv
21      import logging
22      from typing import List, Optional, Tuple
23
24      from backend.eval.annotations import parse_timestamp
25
26      logger = logging.getLogger(__name__)
27
28      Interval = Tuple[float, float]
29
30      _START_NAMES = ("start", "start_time")
31      _END_NAMES = ("end", "end_time")
32
33
34      def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35          """If `row` is a header naming start/end columns, return their indices, else
None."""
36          cells = [(c or "").strip().lower() for c in row]
37          start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
38          end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
39          if start_idx is not None and end_idx is not None:
40              return start_idx, end_idx
41          return None
42
43
44      def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45          """Load GT rally intervals from either golden-CSV convention.
46
47          strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48          (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49          strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50          skipped ? a silently shrinking GT set flatters precision.
51
52          Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53          (they break one-to-one matching and usually indicate a labeling error).
54          """
55          intervals: List[Interval] = []
56          skipped = 0
57          cols: Optional[Tuple[int, int]] = None   # (start_idx, end_idx); None until detected
58          saw_header = False
59
60          with open(csv_path, newline="", encoding="utf-8-sig") as f:
```

```
61              for lineno, row in enumerate(csv.reader(f), start=1):
62                  if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                      continue
64                  if cols is None and not saw_header:
65                      header = _header_columns(row)
66                      if header is not None:
67                          cols = header
68                          saw_header = True
69                          continue            # consume the header row
70                      cols = (0, 1)           # headerless: positional start,end
71                  assert cols is not None     # every branch above assigns it
72                  si, ei = cols
73                  try:
74                      if len(row) <= max(si, ei):
75                          raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
76                      start = parse_timestamp(row[si])
77                      end = parse_timestamp(row[ei])
78                      if start < 0:
79                          raise ValueError(f"negative start ({start})")
80                      if start >= end:
81                          raise ValueError(f"start ({start}) >= end ({end})")
82                  except ValueError as e:
83                      if strict:
84                          raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85                      skipped += 1
86                      continue
87                  intervals.append((start, end))
88
89          if skipped:
90              logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92          intervals.sort(key=lambda iv: iv[0])
93          for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94              if s2 < e1:
95                  logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                                 "labeling error? Overlaps break one-to-one matching.",
97                                 csv_path, s1, e1, s2, e2)
98          return intervals
99
```

14. user (2026-06-20T10:03:53.592Z)

```
1       # training/ ? the owned-model training tree (ADR-008)
2
3       Everything that TRAINS models lives here; everything that SERVES them lives in
`backend/`.
4       The boundary between the two is an **artifact bundle** (weights + manifest), never a
code
5       import. This tree is the pre-staged contents of the future `sports-temporal-models` repo
?
6       the split happens when the ADR-008 triggers fire (Gen-2 / ms-swift / cloud training, or
a
7       second artifact consumer), and by design it will be a file move, not a refactor.
8
9       ## The wall (CI-enforced, see `tests/test_import_smoke.py`)
10
11      1. **`backend/` may NEVER import `training.*`.** Serving consumes versioned artifact
12         bundles only. The import-smoke suite asserts `import backend.main` loads no
13         `training.*` module.
14      2. **Feature extraction is NOT defined here.** Train == serve by construction:
15         both sides import `backend/features/` (pure numpy/scipy). If a training experiment
16         needs a feature change, change it THERE and bump `FEATURE_SCHEMA_VERSION`.
17      3. **Heavy/training-only dependencies go in `requirements-train.txt`**, never in the
```

```
18         root `requirements.txt`.
19      4. **Tests use synthetic fixtures only** until the M0.1 consent ledger exists ? no real
20         trajectories or labels are ever committed here.
21      5. **Weights are never committed** (gitignored; distribute via private GitHub Releases).
22         The committed, reviewable surface is the artifact **manifest**.
23
24      ## Layout
25
26      - `gen0/` ? the Gen-0 TAL harness (M0.4): one-command train ? eval (honest LOVO) ?
27        ship-gate over the golden corpus. See `gen0/README` notes in the module docstrings.
28      - `requirements-train.txt` ? training-only deps.
29
30      ## Ship-gate ownership
31
32      The trainer runs the gate as pre-flight, but the **product side owns the judge**
33      (`backend/eval/regression.py` + `metrics.match_intervals` + the canonical GT loader):
34      no model admits itself to serving. The gate refuses commercial ship while the WASB C3
35      checkpoint-provenance blocker is open, regardless of F1.
36
```

15. user (2026-06-20T10:03:54.011Z)

```
1       # 02 · Compute map & data contract
2
3       **Status:** research synthesis, code-verified 2026-06-20 · **Date:** 2026-06-20
4
5       > What actually consumes GPU/CPU/WSL/external-API across the three repos, ranked by how
hard each is to
6       > lift into a Linux cloud-GPU container ? plus the one data contract everything keys on.
7
8       ## 1. Compute footprint of `badminton-highlight-indexer` (the consumer/trainer)
9
10      | Component | Needs | Portability risk ? cloud Linux GPU |
11      |---|---|---|
12      | **`gemini` segmenter** (default) | external paid API (`GEMINI_API_KEY`) | **LOW.**
Pure HTTP; no GPU. Silently returns **zero** segments without the key ? inject as a secret. |
13      | **`motion` segmenter** | CPU (OpenCV frame-diff) | **NONE.** Pure CPU, no API key. |
14      | **`yolo_hybrid` segmenter** | GPU (torchvision Faster R-CNN + ByteTrack); auto CPU-
fallback | **LOW.** Pure-pip; runs on CPU or CUDA. Pin `supervision<0.30` (ByteTrack API
drifts). Pre-bake COCO weights into the image to avoid first-run hub pulls. |
15      | **`trajectory_hybrid` / TrackNet / WASB** | **GPU + WSL2 bridge** (TensorFlow);
hardcoded `~/models` paths, hardcoded `device='cuda'` (no CPU fallback); **un-vendored**
research repos; TrackNet healthcheck requires a visible GPU | **HIGH ? but the swap seam is
already built.** Runs *only* via `wsl -d Ubuntu ? conda activate` **on Windows** ? on a Linux
GPU box WSL is **dropped, not ported** (the conda env runs natively). The `DetectorRunner` ABC
([`detectors/base.py`](../../backend/pipeline/detectors/base.py)) exists *expressly* to de-risk
this: WSL plumbing is a separate `WslCommandMixin` (a native runner must not inherit it),
Windows behavior is *"100% preserved,"* and the **parity contract is the `Frame,Visibility,X,Y`
CSV** ? any runner emitting it reuses `parse_trajectory_csv` + `trajectory_to_action_windows`
verbatim, so everything downstream is byte-identical. The `LinuxNativeRunner` impl doesn't exist
yet / has never been run ? prove it via the **M3.5 parity test on DigitalOcean** ([04](04-BUILD-
AND-TEST-PLAN.md)). |
16      | **`fusion_hybrid` / Gen-0 head** | CPU/tiny GPU (numpy/scipy) | **NONE.**
[`training/gen0/harness.py`](../../training/gen0/harness.py): ~1M params, ~4.5 GB VRAM, trains
in *minutes*. This is the cheap retrain target. |
17      | **`ffmpeg` chunking / stitching** | CPU (`libx264`) | **LOW.** No nvenc/driver-codec
dependency (a portability win) ? `apt-get install ffmpeg` is enough. |
18      | **Gen-2 VLM** (ms-swift / QLoRA Qwen3-VL) | **multi-GPU** (6?8-GPU GRPO reference; 24
GB VRAM floor) | **OUT OF SCOPE for $200.** *"Largest open risk, currently unquantified."* Do
not attempt on this budget. |
19
20      **Two structural container blockers beyond WSL:**
21      - **TF-vs-torch runtime conflict.** TrackNet is TensorFlow; the rest is PyTorch. Their
pip-bundled CUDA
22        userspaces disagree ? *"works alone, segfaults together."* Fix = **two single-
```

framework images** (a torch
23   image, a TF image) sharing a `/scratch` volume. So the container topology itself is
non-trivial.
24   - **CUDA wheel selection.** `torch>=2.12.0` pinned with no index ? the
Dockerfile/installer must pick
25   `--index-url ?/cu124` vs `?/cpu` explicitly.
26
27   **Path / I/O blockers (confirmed live):**
28   - Hardcoded literal `"output"` in
[`trajectory_hybrid.py:196,272`](../../backend/pipeline/segmenters/trajectory_hybrid.py) ? not
redirected by `--output-dir`.
29   - Local-file assumptions: `cv2.VideoCapture`, `ffmpeg -i <path>`, whole-file MD5,
`allowed_video_root`
30   check in [`main.py`](../../backend/main.py). No URL/stream path ? needs a
`fetch(uri)?local` + `put` shim.
31
32   ## 2. Compute footprint of `sports-data-collector` (the producer/curator)
33
34   CPU-first; *"the gap is narrow."* Only GPU touchpoint is **opt-in `nvenc`** (`--encoder
nvenc`, commit #28)
35   for proxy transcode + batch transcode ? defaults to CPU `libx264`. Shares the same
local-filesystem + MD5
36   assumptions. The export **`manifest.json` omits an `md5` field** (`proxy.py` carries
only `source_md5` /
37   `proxy_md5`); the collector keys by **human `video_id`** (`curate.py`), while the
indexer keys by **file
38   MD5** ? see §4.
39
40   ## 3. Compute footprint of `rally-annotator` (the labeler)
41
42   Effectively **no heavy compute** ? a VLC 3.x Lua extension a human drives to mark rally
`start`/`end` +
43   ending reason ? `<video>.rallies.csv`. Nothing to containerize; it's the *source* of the
labels that must
44   land in the bucket.
45
46   ## 4. The one data contract everything keys on
47
48   ```
49   rally-annotator   sports-data-collector   badminton-highlight-indexer
50   (label by hand) ?????????? (curate + license + export) ????
(validate?segment?index?stitch?eval?train)
51   <video>.rallies.csv   SQLite ? manifest.json + CSVs   keyed by file MD5
52   ```
53
54   - **Identity:** the indexer keys videos by **whole-file MD5** (`compute_video_id`, 64
KB-chunked); the
55   collector keys by **human `video_id`**. The manifest's `local_path` ties them
together. **Any re-encode
56   changes the MD5** and silently breaks the join.
57   - **Intervals:** temporal `[start, end)` in **decimal seconds**; ending-reason taxonomy
58   (winner/forced_error/unforced_error/service_fault/let/other) is the cross-sport label
set.
59   - **Train/serve wall (ADR-008):** the boundary between training and serving is an
artifact bundle
60   (weights + manifest), **never a code import** ? keyed by content/semver.
61
62   ### What this means for a bucket workflow
63
64   1. **Add `md5` to the collector export manifest** (or decide *original-vs-proxy* keying)
before feeding
65   a bucket ? otherwise the MD5-keyed consumer can't resolve labels?video. *This is an
M2 prerequisite, not
66   a footnote.*
67   2. **Store byte-identical files** the labels were made against. If you must proxy-

```
transcode for size, carry
68         an explicit `original_md5 ? proxy_md5` mapping so the join survives.
69      3. A cloud `train`/`infer` job's **inputs** = `{video object(s), label CSV(s),
manifest.json}` from the
70         bucket; its **outputs** = `{versioned weight bundle}` (train) or `{highlights.mp4,
intervals.csv,
71         report.json}` keyed by `video_id` (infer). These map cleanly onto the existing per-
video output layout.
72
73      ### Inputs/outputs a cloud job must exchange (summary)
74
75      | Verb | Pulls from bucket | Pushes to bucket |
76      |---|---|---|
77      | `train` | `videos/`, `labels/`, `manifest.json` (+ latest checkpoint to resume) |
`weights/<semver>/{model, manifest}` + periodic `weights/checkpoints/` |
78      | `infer` | one `videos/<md5>.mp4` | `outputs/<video_id>/{highlights.mp4, intervals.csv,
report.json}` |
79
```

16. assistant (2026-06-20T10:03:57.176Z)

Excellent, this is the contract doc. Now let me read the SETUP_NEW_MACHINE doc, the synth_trajectory_from_labels function, and the test_worker.py to confirm the exact commands and schema in practice.

17. user (2026-06-20T10:03:58.105Z)

```
1       # Setting up a new machine ? zero ? training & inference
2
3       **The canonical "spin up a box and make the models ready" runbook.** Copy-pasteable;
followable by
4       any human. DigitalOcean is the worked example (GCP/AWS/Paperspace are analogous ? only
the provision
5       step differs).
6
7       > **Last verified:** 2026-06-20 on a DigitalOcean Droplet (Ubuntu 24.04, 2 vCPU / 3.8
GB, **no GPU**).
8       > Bootstrap succeeded (torch 2.12.1+cpu); full mocked suite **922 passed / 11 skipped**;
a **secret-free**
9       > CPU inference (no GPU, no API key) detected **2 rallies** and persisted them to
SQLite.
10      >
11      > ? Assumes the DECOUPLED_COMPUTE PRs are merged: the CPU-mock detector seam,
`deploy/digitalocean/bootstrap.sh`,
12      > the storage layer, and the worker dispatcher (#251). The **bucket** steps (§7) are
config-driven and
13      > **verified against real DO Spaces + GCS** ? see the appendix.
14
15      ## What you'll have at the end
16
17      A machine that can: run the test suite (green), run **secret-free CPU inference**
(rallies, no GPU/keys),
18      run **real Gemini inference** (with a key), and run **Gen-0 model training** ? plus
pointers to the GPU /
19      native-detector path and bucket-driven train/infer.
20
21      ---
22
23      ## 1. Credentials to create once (least-privilege)
24
25      | # | Credential | Where | Needed for |
26      |---|---|---|---|
27      | 1 | **SSH keypair** (public key added to the provider) | `ssh-keygen -t ed25519 -f
~/.ssh/rally` ? paste `.pub` into DO ? Settings ? Security | logging into the box |
28      | 2 | **DigitalOcean API token** (read+write) | DO ? API ? Tokens | `doctl` provisioning
```

(optional ? you can click-create) |
29 | 3 | **GitHub repo read access** | a read-only **deploy key** *or* a fine-grained
PAT (`Contents: Read`) | `git clone` on the box (or use the `scp` path in §3) |
30 | 4 | **`GEMINI_API_KEY`** | Google AI Studio / Vertex | **real** Gemini inference only
(§6b). The secret-free path (§6a) needs none. **Rotate before sharing.** |
31 | 5 | **Bucket creds** ? DO Spaces key/secret, or GCS SA JSON, or Drive SA JSON + folder
| provider console | bucket-driven train/infer (§7, *pending PR-B*) |
32 | 6 | **GPU box** (Paperspace free M4000 / GPU Droplet) | provider console | real
WASB/TrackNet detectors + faster runs (§8) |
33
34   **Secret hygiene:** never put a secret in the repo, a PR, or `config.json`. Set them as
env vars on the box
35   (or a git-ignored `.env`). Scope each as narrowly as the UI allows and rotate/revoke
when done.
36
37   ---
38
39   ## 2. Provision the machine
40
41   > **? GPU on GCP now, not DigitalOcean (2026-06-20, ADR-010).** DO GPU Droplets are
42   > North-America-only (`nyc2`/`tor1`/`atl1`) ? there is **no Singapore/India DO GPU**, so
the GPU box +
43   > co-located bucket moved to **GCP `asia-south1` (Mumbai)**. For the GPU path follow
`deploy/gcp/README.md`
44   > (bucket `gs://khelsutra-rally-corpus`, native WASB runner `detector_impl="native"`,
`$100` budget +
45   > teardown). The DigitalOcean steps below remain valid only for a **CPU** box / the
historical stub e2e.
46
47   - **CPU box** (suite + secret-free stub inference + Gen-0 training plumbing): cheapest
Ubuntu 24.04
48   Droplet, **2 vCPU / 4 GB**, region nearest you, **attach the SSH key** (#1). ~$0.06/hr
? destroy when done.
49   - **GPU box** (real WASB/TrackNet detectors ? real trajectory extraction + training): a
DO GPU Droplet.
50   ? You **cannot add a GPU to a CPU Droplet** ? it's a separate machine type, in **GPU
regions**
51   (`nyc2`/`tor1`/`atl1`, **not** `blr1`), with **dual local NVMe**. **RTX 4000 Ada (20
GB, ~$0.76/hr)** is
52   plenty for WASB + Gen-0; the `gpu-*-base` image preinstalls the NVIDIA driver. **Co-
locate the Space in the
53   GPU box's region** for cheap corpus-pull.
54
55   CLI provision (after `doctl auth init` with token #2 ? **verify size/image slugs first**
via `doctl compute size list`):
56   ```bash
57   # CPU box:
58   doctl compute droplet create rally-box --image ubuntu-24-04-x64 --size s-2vcpu-4gb \
59   --region blr1 --ssh-keys <your-key-fp> --wait
60   # GPU box (RTX 4000 Ada):
61   doctl compute droplet create rally-gpu --image gpu-h100x1-base --size gpu-4000adax1-20gb
\
62   --region tor1 --ssh-keys <your-key-fp> --wait
63   doctl compute droplet list   # note the IP -> ssh -i ~/.ssh/rally root@<IP>
64   ```
65
66   ---
67
68   ## 3. Get the code onto the box
69
70   **Option A ? git clone** (private repo, so authenticate with #3):
71   ```bash
72   ssh -i ~/.ssh/rally root@<IP>
73   git clone git@github.com:avidullu/badminton-highlight-indexer.git ~/rally   # via deploy
key

```
#   or: git clone https://<PAT>@github.com/avidullu/badminton-highlight-indexer.git
~/rally
cd ~/rally
```

Option B ? push a local checkout (no GitHub access needed; what the maintainer
used):
```bash
# on your machine, from the repo root:
git archive --format=tar.gz -o /tmp/rally.tgz HEAD
scp -i ~/.ssh/rally /tmp/rally.tgz root@<IP>:/root/
ssh -i ~/.ssh/rally root@<IP> 'mkdir -p ~/rally && tar -xzf ~/rally.tgz -C ~/rally'
```

---

4. Bootstrap (one command)

```bash
cd ~/rally
./deploy/digitalocean/bootstrap.sh          # CPU box
#   or
./deploy/digitalocean/bootstrap.sh --gpu     # GPU box (installs CUDA torch)
```

It mirrors CI: installs `ffmpeg` + opencv runtime, creates `.venv`, installs CPU-or-CUDA
`torch`, then
`pip install -e .[dev]`, and prints a device check (`cuda_available True/False`).

<details><summary>Manual equivalent (if you can't run the script)</summary>

```bash
sudo apt-get update
sudo apt-get install -y python3 python3-venv python3-pip git ffmpeg libgl1
sudo apt-get install -y libglib2.0-0t64 || sudo apt-get install -y libglib2.0-0
python3 -m venv .venv && . .venv/bin/activate && python -m pip install -U pip
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu   # or
.../whl/cu124 on GPU
pip install -e .[dev]
```
</details>

---

5. Verify the machine (the smoke gate)

```bash
. .venv/bin/activate
python -m pytest -q          # expect all green (922 passed / 11 skipped on the current
tree)
```
A green suite proves the whole pipeline runs on this box with **no GPU, no WSL, no
bucket, no API key**.

---

6. Run inference

6a. Secret-free CPU inference (no GPU, no API key) ? the "mock a GPU with a CPU"
path

Drop a permissive smoke `config.json` in the repo root (the CLI reads `./config.json`
from the cwd):
```bash
cd ~/rally && cat > config.json <<'JSON'
```

```
    {
      "sport": "badminton",
      "indexing": { "default_segmenter": "wasb_hybrid", "detector_impl": "stub",
"skip_ai_handoff": true },
      "validation": { "min_fps": 0, "min_blur_threshold": 0, "max_scene_cuts_per_minute":
1000,
                      "relevance": { "court_pixels_min_ratio": 0.0 } }
    }
    JSON

    # a throwaway 12s 1280x720 test clip
    ffmpeg -y -f lavfi -i testsrc=duration=12:size=1280x720:rate=30 -pix_fmt yuv420p
/root/sample.mp4

    # run ? env -u drops any Gemini key to prove none is needed
    env -u GEMINI_API_KEY RALLY_DETECTOR_IMPL=stub \
      indexer --video-path /root/sample.mp4 --segmenter wasb_hybrid --output-dir ~/rally/out
    ```
    Expect: `detector_impl=stub ? CPU mock runner`, `Phase 3 SKIPPED (fully-local)`, and
    `Indexed 2 play segments` in `~/rally/out/sports_indexer.db`. *(`detector_impl=stub`
swaps the GPU+WSL
    detector for a deterministic CPU one; `skip_ai_handoff=true` makes candidate windows the
rallies, so no
    Gemini handoff happens.)*

    ### 6b. Real inference (Gemini segmenter)

    ```bash
    export GEMINI_API_KEY="<your-rotated-key>"          # secret stays in the env, never in
the repo
    # use a real badminton clip (must pass validation: ?1280x720, ?30fps, a green court,
etc.)
    indexer --video-path /root/match.mp4 --segmenter gemini --output-dir ~/rally/out
    #   or the hybrid: --segmenter wasb_hybrid (WASB needs a GPU box, §8) with
skip_ai_handoff=false
    ```

    ## 7. Object storage ? bucket-driven train/infer (S3-compatible / GCS / Drive)

    The worker (`python -m backend.worker`) reads/writes by **URI**, so you choose a backend
with **config +
    credentials ? no code change**. A bare path / `local://` is a passthrough (today's
behaviour); `s3://` and
    `gcs://` go to a bucket. Install only the extra you use (cloud SDKs are lazy-imported).

    > **Verified end-to-end 2026-06-20:** the worker bucket round-trip (pull video ? infer ?
push outputs) passed
    > against **real DigitalOcean Spaces** (sgp1), a real **GCS** client (emulator), and
MinIO.

    ### 7a. S3-compatible ? AWS S3 / DigitalOcean Spaces / Cloudflare R2 / MinIO

    One backend for all four ? they differ only by `endpoint_url` + region. `pip install -e
.[storage-s3]`.

    Credentials ? `~/.aws/credentials` (boto3's default chain; **never** in the repo):
    ```ini
    [default]
    aws_access_key_id = <key>
    aws_secret_access_key = <secret>
    ```
    Config ? `config.json` `storage.s3`:
    ```jsonc
```

```
183     "storage": { "backend": "local", "s3": {
184       "endpoint_url": "https://sgp1.digitaloceanspaces.com",  // DO Spaces (VERIFIED);
region = the Space's, e.g. sgp1
185       "region": "sgp1"
186       // AWS S3 : omit endpoint_url; set region (e.g. ap-south-1)
187       // R2     : endpoint_url https://<account>.r2.cloudflarestorage.com ; region "auto"
188       // MinIO  : endpoint_url http://host:9000 ; region us-east-1 ; "addressing_style":
"path"
189     }}
190     ```
191     **Run** (secret-free flavor; use `--segmenter gemini` for real detection):
192     ```bash
193     env -u GEMINI_API_KEY RALLY_DETECTOR_IMPL=stub python -m backend.worker infer \
194       --video-uri s3://<bucket>/videos/clip.mp4 --out-uri s3://<bucket> \
195       --segmenter wasb_hybrid --set indexing.skip_ai_handoff=true
196     ```
197
198     ### 7b. Google Cloud Storage
199
200     `pip install -e .[storage-gcs]`. **Credentials** ? a service-account JSON (SA with
*Storage Object Admin*):
201     ```bash
202     export GOOGLE_APPLICATION_CREDENTIALS=/path/to/sa.json
203     ```
204     **Config** ? `storage.gcs.project` (no endpoint ? default `googleapis.com`):
205     ```jsonc
206     "storage": { "backend": "local", "gcs": { "project": "<gcp-project-id>" } }
207     ```
208     Same `worker infer` command with `gcs://<bucket>/?` URIs. *(On-box test with no GCP
creds:
209     `pip install gcp-storage-emulator && gcp-storage-emulator start --port 9023 --in-memory`
+
210     `export STORAGE_EMULATOR_HOST=http://127.0.0.1:9023`.)*
211
212     ### 7c. Google Drive *(corpus source)*
213
214     **On a machine WITH Google Drive for Desktop (the simplest path ? no new code):** the
mount exposes
215     Drive as a **real filesystem path**, so just pass that path ? the default `local`
backend handles it
216     (identity passthrough; Drive streams + caches on access). Windows: `G:\My Drive\?`;
macOS:
217     `~/Library/CloudStorage/GoogleDrive-<acct>/My Drive/?`. The shared golden-corpus folder
218     `drive/folders/1sHGL7iacnmM80ChT-p2cKy638gK504T_` maps to
219     `G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June 15\`, so:
220     ```bash
221     python -m backend.worker infer --video-uri "G:\My Drive\?\[Done] Video 4 -
Badminton\GX020094_proxy.mp4" ?
222     ```
223     No service account, no API, no extra install. **A headless GPU VM has no mount** ? use
this on the
224     workstation (e.g. to copy Drive?bucket), and a bucket/`gdrive://` URI on the VM.
225
226     **On a HEADLESS box (no mount) ? the `gdrive://` API backend:** `pip install -e
.[storage-gdrive]`.
227     **Credentials** ? a service-account JSON; the SA's email must be granted access to the
file/folder
228     (share it in the Drive UI ? an SA sees nothing by default; true multi-tenant "point at
*your* Drive"
229     needs OAuth, not an SA). The path comes from the env, never config:
230     ```bash
231     export GDRIVE_SERVICE_ACCOUNT_JSON=/path/to/sa.json   # or reuse
GOOGLE_APPLICATION_CREDENTIALS
232     ```
233     Drive has no path namespace ? objects are addressed by an opaque **id**:
```

```text
gdrive://<file-id>            # one file  (worker --video-uri, stat, delete)
gdrive://<folder-id>          # list a folder's immediate children
gdrive://<folder-id>/<title>  # upload target (put creates <title> in the folder)
```
*(Quick one-off pull without the backend: `gdown <file-id> -O clip.mp4`, then pass the local path.
For the GPU run, prefer staging Drive?GCS once and reading `gcs://`.)*

Bucket layout (for the bucket backends): `videos/ labels/ manifest.json weights/ outputs/<video_id>/`
(see `docs/DECOUPLED_COMPUTE/04`).

---

8. Run training (Gen-0 rally head)

CPU-runnable, minutes ? but it needs a **labeled corpus** (videos + `.rallies.csv` labels + a manifest),
which a fresh box doesn't have. Bring it via §7's bucket (`worker train`, pending) or `scp` it up, then:
```bash
python -m training.gen0.harness --manifest <manifest.json> --floor <baseline> --version 0.1.0
```
The harness runs train ? eval ? ship-gate and emits a versioned weight bundle. *(The Gen-2 VLM is a
separate, GPU-heavy, separately-budgeted effort ? out of scope here.)*

---

9. GPU box ? port procedure (NVMe + native WASB)

The GPU box is **cattle, not a pet**: durable state lives in the bucket + git, so a fresh box is running in
minutes. Port = provision (§2) ? code on (§3) ? NVMe ? `bootstrap.sh --gpu` ? `gpu_setup.sh` ? pull corpus.

```bash
# on the box, after §3 put the code at ~/rally:
cd ~/rally
sudo bash deploy/digitalocean/mount_scratch.sh && export TMPDIR=/scratch   # detect+mount the spare NVMe
./deploy/digitalocean/bootstrap.sh --gpu                                    # CUDA torch; expects cuda_available True
. .venv/bin/activate && pip install -e .[storage-s3]
bash deploy/digitalocean/gpu_setup.sh                                       # native WASB conda env (no WSL) ? M3.5 enabler
# secrets (the only two non-reproducible items): Gemini key + bucket creds
mkdir -p /root/.keys && printf '%s' '<rotated-key>' > /root/.keys/GEMINI_API_KEY && chmod 600 /root/.keys/GEMINI_API_KEY
install -m600 /dev/stdin ~/.aws/credentials <<'INI'
[default]
aws_access_key_id = <key>
aws_secret_access_key = <secret>
INI
```

- **NVMe scratch** (`mount_scratch.sh`): DO GPU Droplets ship a second raw NVMe; the script formats+mounts it
  at `/scratch` and the pipeline's sync-first decode runs there (`export TMPDIR=/scratch` or `--scratch /scratch`).
  Ephemeral, not in fstab. On a CPU box it just makes `/scratch` on the boot disk.
- **Native WASB** (`gpu_setup.sh`): replicates `docs/TRACKNET_WSL_SETUP.md`'s conda env **natively on Linux**

```
284       (no WSL) ? the M3.5 enabler. ? Needs `wasb_badminton_best.pth.tar` present and the
**WASB-C3** weight
285       provenance cleared before sharing any derived model.
286     - **The detector itself:** real `--segmenter wasb_hybrid` extraction needs the Linux-
native `DetectorRunner`
287       (tracked M3.5, `docs/DECOUPLED_COMPUTE/04`). Until it lands, validate the train/infer
plumbing with
288       `detector_impl=stub`.
289     - **Teardown** = `doctl compute droplet delete rally-gpu`. Re-spin tomorrow identically.
290
291     ---
292
293     ## 10. Teardown
294
295     ```bash
296     rm -f ~/rally/config.json ~/sample.mp4        # remove the throwaway smoke artifacts
297     doctl compute droplet delete rally-box        # or destroy in the console
298     ```
299     Revoke any temporary token/key (#2?#5) and delete the SSH key from the provider when the
box is gone.
300
301     ---
302
303     ## Appendix ? verification log
304
305     | Step | Command | Result (2026-06-20, DO Ubuntu 24.04, 2 vCPU, no GPU) |
306     |---|---|---|
307     | Bootstrap | `bootstrap.sh` | ? torch 2.12.1+cpu, deps + editable build |
308     | Smoke | `pytest -q` | ? 922 passed, 11 skipped (49s) |
309     | Secret-free infer (local) | §6a | ? 2 rallies, persisted to SQLite, `GEMINI_API_KEY`
unset |
310     | **S3 worker round-trip ? real DO Spaces (sgp1)** | §7a | ? pulled
`s3://rally-e2e/videos/?` ? 2 rallies ? pushed `outputs/<md5>/intervals.csv` |
311     | **GCS worker round-trip** (emulator) | §7b | ? pulled `gcs://?` ? 2 rallies ? pushed
`gcs://?/outputs/<md5>/?` |
312     | S3 backend vs MinIO (put/get/list/stat + presigned GET) | §7a | ? |
313
```

18. user (2026-06-20T10:03:58.395Z)

```
1       """CPU stub shuttle-trajectory runner ? "mock a GPU with a CPU".
2
3       A ``DetectorRunner`` that produces a deterministic ``Frame,Visibility,X,Y`` trajectory
4       CSV with **no GPU, no WSL, no model weights** ? so the whole trajectory-hybrid pipeline
5       (``wasb_hybrid`` / ``tracknet_hybrid``) can run end-to-end on a CPU-only box (CI, a CPU
6       container, a laptop) for regression testing. The person/yolo path already CPU-falls-back
7       (``use_gpu=False``); this closes the last GPU/WSL-only gap so the *entire* pipeline is
8       runnable ? and regression-gateable ? without a GPU.
9
10      Two modes:
11        * **replay** ? emit an existing ``Frame,Visibility,X,Y`` CSV verbatim (drive the
12          pipeline with a committed/golden trajectory; deterministic + cross-OS).
13        * **synth** (default) ? generate a deterministic synthetic trajectory of
14          ``num_rallies`` in-flight bursts (visible + fast-moving frames) separated by dead
15          gaps, so windowing yields a stable, known number of candidate windows.
16
17      This is the CPU seam the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
18      "Optional Linux-native path") and the **seed of the future ``LinuxNativeRunner``**
19      (M3.5, ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``). It is selected via
20      ``indexing.detector_impl="stub"`` or the ``RALLY_DETECTOR_IMPL=stub`` env var; the real
21      WSL runners stay the default, so production behaviour is unchanged.
22      """
23      import os
24      import csv as _csv
25      import shutil
```

```
    import logging
    from dataclasses import dataclass
    from typing import Any, Dict, List, Optional, Tuple

    from backend.pipeline.detectors.base import DetectorRunner
    # Reuse the model-agnostic CSV parsing + windowing (no torch/tf pulled here).
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    logger = logging.getLogger(__name__)


    @dataclass
    class StubConfig:
        """Settings for the CPU stub runner (built from ``config.json`` ->
``indexing.stub``)."""
        fps: float = 30.0
        frame_width: float = 1280.0
        num_rallies: int = 2
        rally_seconds: float = 4.0
        gap_seconds: float = 3.0          # dead gap between rallies (> merge_gap so
windows don't merge)
        lead_seconds: float = 2.0         # dead lead-in
        step_px: float = 12.0             # per-frame shuttle displacement during a rally
(>> velocity_thresh)
        replay_csv: Optional[str] = None   # if set + exists, emit this CSV verbatim instead
of synthesising

        @classmethod
        def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "StubConfig":
            s = (idx_cfg or {}).get("stub", {}) or {}
            return cls(
                fps=float(s.get("fps", 30.0)),
                frame_width=float(s.get("frame_width", 1280.0)),
                num_rallies=int(s.get("num_rallies", 2)),
                rally_seconds=float(s.get("rally_seconds", 4.0)),
                gap_seconds=float(s.get("gap_seconds", 3.0)),
                lead_seconds=float(s.get("lead_seconds", 2.0)),
                step_px=float(s.get("step_px", 12.0)),
                replay_csv=s.get("replay_csv"),
            )


    class StubDetectorRunner(DetectorRunner):
        """Deterministic CPU trajectory runner (no GPU/WSL). See the module docstring."""

        def __init__(self, cfg: Optional[StubConfig] = None):
            self.cfg = cfg or StubConfig()

        def healthcheck(self) -> Tuple[bool, str]:
            return True, "stub CPU detector (no GPU/WSL)"

        def _rows(self) -> List[Tuple[int, int, float, float]]:
            """Deterministic ``(frame, visible, x, y)`` rows: ``num_rallies`` in-flight
bursts
            separated by dead gaps. During a rally the shuttle oscillates ``step_px``/frame
?
            well above the default 0.02 velocity threshold ? so every rally frame is
"active"
            and each burst becomes exactly one candidate window."""
            cfg = self.cfg
            rows: List[Tuple[int, int, float, float]] = []
            frame = 0
            y = cfg.frame_width / 4.0
            base_x = cfg.frame_width / 2.0
            for _ in range(int(cfg.lead_seconds * cfg.fps)):
```

```
                rows.append((frame, 0, 0.0, 0.0))
                frame += 1
            for r in range(cfg.num_rallies):
                for i in range(int(cfg.rally_seconds * cfg.fps)):
                    x = base_x + (cfg.step_px if i % 2 else -cfg.step_px)
                    rows.append((frame, 1, x, y))
                    frame += 1
                if r != cfg.num_rallies - 1:
                    for _ in range(int(cfg.gap_seconds * cfg.fps)):
                        rows.append((frame, 0, 0.0, 0.0))
                        frame += 1
            return rows

        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Write a deterministic trajectory CSV and return its path (never touches the
GPU/WSL)."""
            os.makedirs(output_win_dir, exist_ok=True)
            stem = os.path.splitext(os.path.basename(str(video_win_path).replace("\\",
"/")))[0]
            key = f"{stem}__{video_id[:12]}" if video_id else (stem or "stub")
            out_csv = os.path.join(output_win_dir, f"{key}_stub.csv")

            if self.cfg.replay_csv and os.path.exists(self.cfg.replay_csv):
                shutil.copyfile(self.cfg.replay_csv, out_csv)
                logger.info("Stub runner: replayed trajectory %s -> %s",
self.cfg.replay_csv, out_csv)
                return out_csv

            rows = self._rows()
            with open(out_csv, "w", newline="") as f:
                w = _csv.writer(f)
                w.writerow(["Frame", "Visibility", "X", "Y"])
                for frame, vis, x, y in rows:
                    w.writerow([frame, vis, f"{x:.3f}", f"{y:.3f}"])
            logger.info("Stub runner: wrote %d synthetic trajectory rows -> %s", len(rows),
out_csv)
            return out_csv

        # Model-agnostic CSV parsing + windowing ? identical to the real runners.
        parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
        trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)


    def synth_trajectory_from_labels(labels_csv: str, out_csv: str, *, fps: float = 30.0,
                                     frame_width: float = 1280.0, tail_pad: float = 2.0,
                                     step_px: float = 12.0) -> str:
        """CPU "mock GPU" for TRAINING: write a deterministic Frame,Visibility,X,Y
trajectory whose
        in-flight (visible + oscillating) frames cover each labeled rally window and whose
dead frames
        cover the gaps ? so the synthetic trajectory *encodes* the rally labels.

        This lets the Gen-0 harness train + leave-one-video-out eval end-to-end on coherent
data with
        no GPU/WSL, validating the whole training pipeline before the real WASB detector
lands. (The
        learned model is meaningful for plumbing, not for production ? the trajectory is
synthetic.)
        """
        rallies: List[Tuple[float, float]] = []
        with open(labels_csv, newline="", encoding="utf-8") as f:
            for row in _csv.DictReader(f):
                try:
```

```
                    rallies.append((float(row["start_time"]), float(row["end_time"])))
                except (KeyError, ValueError):
                    continue
        duration = (max(e for _, e in rallies) + tail_pad) if rallies else 10.0
        nframes = max(1, int(duration * fps))
        base_x = frame_width / 2.0
        y = frame_width / 4.0
        os.makedirs(os.path.dirname(out_csv) or ".", exist_ok=True)
        with open(out_csv, "w", newline="", encoding="utf-8") as f:
            w = _csv.writer(f)
            w.writerow(["Frame", "Visibility", "X", "Y"])
            for i in range(nframes):
                t = i / fps
                if any(s <= t < e for s, e in rallies):
                    x = base_x + (step_px if i % 2 else -step_px)
                    w.writerow([i, 1, f"{x:.3f}", f"{y:.3f}"])
                else:
                    w.writerow([i, 0, "0.000", "0.000"])
        return out_csv

```

19. assistant (2026-06-20T10:04:01.783Z)

Now I have the full picture. The synth path keys on `row["start_time"]`/`row["end_time"]` from DictReader. Let me confirm the gating/M3.5 status from the BUILD-AND-TEST plan and HANDOFF, and verify the worker train test to confirm what's actually exercised.

20. user (2026-06-20T10:04:03.208Z)

```
1       # 04 · Build & test plan
2
3       **Status:** PROPOSAL · **Date:** 2026-06-20
4
5       > A staged plan a solo dev can stop at any milestone with a working, tested artifact. It
ships the
6       > **container-clean path first** (Gemini + torchvision + motion + Gen-0) and defers the
WSL-bound
7       > WASB/TrackNet detectors until a Linux-native runner exists ? those two are the only
HIGH-risk components
8       > in the stack (see [02](02-COMPUTE-MAP-AND-CONTRACT.md)).
9
10      ## 1. Target architecture (diagram-in-words)
11
12      ```
13         CONTROL PLANE (thin, cheap; already shipped as the async job model)
14         - validates API key, accepts {job_type, uri}; returns 202 + job_id; GET /jobs/{id}
15         - holds NO video bytes; enqueues, polls
16                     ? job descriptor (bucket keys)         ? status / cost
17                     ?                                       ?
18         COMPUTE WORKER (rented GPU; container OR venv; pay GPU-seconds)
19         - ENTRYPOINT dispatcher: mode ? {train, infer}
20         - sync-first: rclone cp bucket ? /scratch (local NVMe); decode/detect on GPU
21         - write artifacts ? /scratch ? upload to bucket; checkpoint train ?30 min
22         - record usage (gpu_seconds, egress, gemini_calls) ? cost ledger (05)
23                     ? pull inputs                           ? push outputs
24                     ?                                       ?
25         OBJECT-STORE BUCKET (data plane ? bytes never via the control plane)
26         videos/   labels/   manifest.json   weights/   outputs/
27      ```
28
29      The control plane coordinates **metadata + jobs**; the **heavy bytes + GPU work live in
the worker**; the
30      **bucket is the only shared state**. This is the control/data-plane split
[`PLATFORM_ARCHITECTURE.md`](../PLATFORM_ARCHITECTURE.md)
31      already names ? what's new is the container/VM image, the real bucket, the rented GPU,
```

and the cost ledger.

2. The two entrypoints (one dispatcher; container or `run.sh` on a VM)

`train` ? *bucket of videos + labels ? versioned weights back to the bucket:*
1. Look for the latest checkpoint under `weights/checkpoints/`; **resume if present** (spot-safety + the firm idempotency/resumability requirement ? a 40-min job must not restart on a blip).
2. `rclone copy` the corpus to `/scratch` (sync-first).
3. Run the **Gen-0 harness as-is** ? already a single self-contained command producing a versioned artifact: `python -m training.gen0.harness --manifest <?> --floor <?> --version <semver>`. Gen-0 needs only numpy/scipy (~4.5 GB VRAM, minutes). **Gen-2 VLM is explicitly out of scope** ([06](06-RISKS-AND-OPEN-QUESTIONS.md)).
4. Run the product-side **ship-gate as pre-flight** ([`backend/eval/regression.py`](../../backend/eval/regression.py) + `metrics.match_intervals`): no model admits itself to serving.
5. Upload weights + manifest to `weights/<semver>/` (ADR-008 artifact bundle, content/semver-keyed). Checkpoint every ?30 min.

`infer` ? *new upload ? segments/highlights ? outputs back to the bucket:*
1. `rclone copyto $VIDEO_URI /scratch/in.mp4` ? **sync-first, not a FUSE random-access decode.**
2. Compute whole-file MD5 ? `video_id` (existing `compute_video_id`). **Store the byte-identical file the labels were made against** ? any re-encode changes the MD5 and breaks the join ([02](02-COMPUTE-MAP-AND-CONTRACT.md)).
3. Run the segmenter (`gemini` default; `yolo_hybrid`/`motion` offline). Gemini path uploads ?500 MB chunks.
4. Write `outputs/<video_id>/{highlights.mp4, intervals.csv (decimal-second [start,end)), report.json}`.
5. Inference is checkpoint-free ? on spot preemption just retry; make the front-end idempotent.

3. Bucket layout

```
rally-bucket/
??? videos/<md5>.mp4              # immutable, byte-identical to what labels were made on
??? labels/<sport>/<video_id>.csv # collector-exported start,end CSVs
??? manifest.json                 # the join table (video_id, csv, local_path, md5, license, fps?)
??? weights/
?   ??? <semver>/model + manifest # train output (artifact bundle)
?   ??? checkpoints/              # spot-resume state (model + optimizer + step)
??? outputs/<video_id>/{highlights.mp4, intervals.csv, report.json}
```

Auth via env/secret (`AWS_ACCESS_KEY_ID/SECRET` for S3/R2/Spaces, or a mounted GCS service-account JSON) ? never baked into the image. Keep `is_commercial`/CURATED training rows physically separated; only those export by default.

4. Milestone ladder (each with a concrete acceptance test)

M0 ? Local CPU smoke test (`$0`, no GPU, no provider).
Build the torch image *or* a venv; run the **committed synthetic, video-free regression fixtures** ([`golden_fixtures.py`](../../backend/eval/golden_fixtures.py)) + motion/yolo_hybrid on a tiny sample, CPU-only (torchvision auto-falls-back).

```
77      *Accept:* `infer --video samples/clip.mp4` produces an `intervals.csv`, and the in-repo
regression subset
78      passes ? no GPU, no WSL. (Runnable in GitHub Actions.)
79
80      **M1 ? Single GPU, manual run.**
81      Launch one GPU (DO Paperspace/Gradient A4000 or GCE Spot L4); run the worker.
82      *Accept:* `nvidia-smi` + `torch.cuda.is_available()==True` inside the worker, and one
`infer` on a real
83      clip completes on GPU, faster than the M0 CPU run.
84
85      **M2 ? Train-from-bucket.**
86      Create the bucket; populate `videos/`, `labels/`, `manifest.json` (**incl. the `md5`
field ? prerequisite,
87      [02](02-COMPUTE-MAP-AND-CONTRACT.md) §4**). Run `train`, pulling via `rclone`, running
Gen-0, pushing a
88      versioned bundle back.
89      *Accept:* `weights/<semver>/` appears in the bucket, the artifact passes the ship-gate
pre-flight, and a
90      re-run **resumes from checkpoint** rather than restarting.
91
92      **M3 ? Inference endpoint on new upload (+ per-video billing).**
93      Deploy as a serverless endpoint (Cloud Run L4 / RunPod Serverless). Control plane
returns `202 + job_id`;
94      worker pulls the upload, writes `outputs/<video_id>/`, and records the cost row
([05](05-COST-ATTRIBUTION.md)).
95      *Accept:* `POST {job_type:"infer", video_uri:"?"}` with a valid API key returns a
`job_id`; polling yields
96      `highlights.mp4` + `intervals.csv` in the bucket **and a per-video cost**; a bad/absent
key is rejected.
97
98      **M3.5 ? WSL?native detector parity, tested on DigitalOcean (the "WSL runs seamlessly"
gate).**
99      Stand up a **`LinuxNativeRunner`** behind the existing `DetectorRunner` ABC
([`detectors/base.py`](../../backend/pipeline/detectors/base.py)
100     ? *"the main de-risk goal"*): it **skips all `wsl`/`conda activate`/staging**, loads the
WASB/TrackNet weights
101     natively (torch/TF), and writes the **identical `Frame,Visibility,X,Y` CSV** so the
model-agnostic
102     `parse_trajectory_csv` + `trajectory_to_action_windows` (and everything downstream ?
windowing, candidates,
103     AI handoff, eval) run **verbatim, byte-for-byte**. On a **DO GPU Droplet / Paperspace
GPU** (Ubuntu +
104     NVIDIA + CUDA) you replicate the `TRACKNET_WSL_SETUP.md` conda env **natively, minus
WSL** ? and since WASB
105     ships pretrained badminton weights (`wasb_badminton_best.pth.tar`), you get a **real
trajectory on DO
106     today**, no TrackNet training needed. Local Windows dev is **untouched** ? the WSL
adapters stay as one
107     backend of the ABC (the base guarantees *"existing behavior is 100% preserved"*); env/OS
selects the runner.
108     *This milestone doubles as the [`DESKTOP_APP_PLAN.md`](../DESKTOP_APP_PLAN.md) P0
compute-feasibility spike.*
109     *Accept (the seamlessness guarantee):* a **parity test** ? the same clip through the
**WSL backend (Windows)**
110     and the **`LinuxNativeRunner` (DO)** yields `Frame,Visibility,X,Y` CSVs that match
within tolerance **and**
111     identical candidate windows; the WASB/TrackNet hybrid segmenter produces the same
rallies on DO as on the
112     WSL box. (WASB and TrackNetV4 are torch vs TF ? keep them as separate runners/images
sharing `/scratch`;
113     never co-install.)
114
115     **M4 ? Scale-to-zero / cost-guard.**
116     Confirm the endpoint scales workers to zero when idle; add a per-key rate limit + a
provider hard spend cap.
```

```
117     *Accept:* after a burst, the dashboard shows **$0 idle GPU billing** between jobs; a
synthetic flood on one
118     leaked key is throttled; the spend alert fires below $200 in a forced-overrun test.
119
120     ## 5. Smallest first PR
121
122     **PR: "Add bucket-aware I/O shim + `train`/`infer` dispatcher + (optional) `rally-torch`
Dockerfile ?
123     CPU-clean path only."** In the indexer repo:
124     1. A thin **storage shim**: `fetch(uri)?local_path` + `put(local_path, uri)` (over
`rclone`/`aws cp`),
125        wired *behind* the existing `resolve_video_path`/`validate_input_path` so local paths
still work unchanged.
126     2. A `run.sh` (or `python -m ?`) **dispatcher** (`mode ? {train, infer}`) calling the
existing CLI entry
127        points ? **no pipeline-logic changes.**
128     3. **De-hardcode** the `"output"` literals in
[`trajectory_hybrid.py:196,272`](../../backend/pipeline/segmenters/trajectory_hybrid.py)
129        to honor an output root (small, isolated).
130     4. *(Optional)* a `Dockerfile` for the torch image (pytorch CUDA 12.4 base, `apt
ffmpeg`, torch from the
131        explicit `--index-url`, `supervision<0.30`, pre-baked COCO weights, `GEMINI_API_KEY`
via env).
132     5. **CI:** run the **video-free regression subset** (on CPU, in a container or venv) ?
this *is* the M0 gate.
133
134     This PR is intentionally WSL-free, GPU-optional, and touches no detector internals ? it
lands the
135     container/bucket seam (the two things missing today) while deferring every HIGH-risk
component to a later,
136     isolated PR.
137
138     ## 6. Container image strategy (when you do containerize)
139
140     - **Image A ? `rally-torch` (ship first; M0?M3):** base
`pytorch/pytorch:2.5.x-cuda12.4-cudnn9-runtime`;
141       `apt ffmpeg`; torch/torchvision from `--index-url ?/cu124`; `supervision<0.30`; pre-
baked COCO weights;
142       `GEMINI_API_KEY` as runtime secret. Covers gemini, motion, yolo_hybrid.
143     - **Image B ? `rally-tracknet-tf` (M4+):** base `tensorflow/tensorflow:2.16-gpu`. This
image is what
144       **kills the WSL bridge** ? inside Linux, TF-GPU runs natively. Lift the TrackNet/WASB
inference out of
145       WSL into an in-container module reading frames from `/scratch`; **vendor** the
research repos + weights;
146       replace hardcoded `device='cuda'` with the Linux-native `DetectorRunner` the ABC
already anticipates;
147       keep the WSL bridge as one backend of a two-backend interface so Windows dev +
existing tests stay green.
148       Torch and TF talk over a shared `/scratch` volume ? **never co-install TF + torch in
one image** (segfault).
149
```

21. user (2026-06-20T10:04:04.564Z)

```
18:    ("true", True), ("False", False), ("3", 3), ("2.5", 2.5), ("wasb_hybrid", "wasb_hybrid"),
26:    wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true",
"indexing.min_segment_duration=1.5",
28:    assert cfg.indexing.skip_ai_handoff is True
61:        "--set", "indexing.skip_ai_handoff=true", "--set", "sport=tennis"])
63:    assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]
122:        "--segmenter", "wasb_hybrid", "--set", "indexing.skip_ai_handoff=true",
147:# --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---
149:def test_cmd_train_orchestration(tmp_path, monkeypatch):
175:    rc = wmain.main(["train", "--corpus-uri", str(tmp_path / "corpus"), "--out-uri",
```

```
str(out_dir),
187:def test_parser_train_detector_args():
189:        "train", "--corpus-uri", "gcs://b", "--out-uri", "gcs://b", "--version", "1.0.0",
190:        "--trajectory-source", "detector", "--segmenter", "fusion_hybrid"])
191:    assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"
194:def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
195:    """--trajectory-source detector pulls videos + runs the resolved DetectorRunner. With
196:    RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the stub IS the runner)."""
200:    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # resolve a CPU stub runner (no
GPU/WSL)
201:    # Fake bytes aren't decodable, so stub the (strict, fail-loud) ffprobe probe to a real
value.
230:    rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
232:                     "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
238:    assert all(s["trajectory"].endswith("_stub.csv") for s in specs)  # the stub runner's
output
241:def test_cmd_train_detector_native_on_tracknet_fails_clean_not_traceback(tmp_path,
monkeypatch):
242:    """detector_impl=native for a family with no native runner must return rc 2 (clean), not
254:    rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(tmp_path /
"out"),
256:                     "--trajectory-source", "detector", "--segmenter", "tracknet_hybrid"])
257:    assert rc == 2  # graceful: _resolve_train_detector caught NotImplementedError, returned
None
261:    """The strict ffprobe probe must return None (caller skips) for a non-video file, never
guess."""
```

22. assistant (2026-06-20T10:04:07.965Z)

I now have everything I need to answer precisely. Let me verify one detail about the `--floor` baseline file existence and the FEATURE_SCHEMA to round out the schema picture, plus confirm the rally-annotator output filename convention.

23. user (2026-06-20T10:04:10.779Z)

```
===floor baselines===
eval_baselines/experiment_leaderboard.json
eval_baselines/gemini_wrapper_2026-06-10.json
eval_baselines/heuristic_lovo_n6.json
eval_baselines/nightly_baseline.json
===rally-annotator output filename in lua===
12:      rally_number,start_time,end_time,ending_reason,sport,shots_count
28:      was loaded, the CSV path fell back to ~/rally_labels.csv and rallies were
31:      provably playing), the tool adopts <video>.rallies.csv and loads any
42:    - New optional "Number of shots" field -> a shots_count column appended to
100:    shortdesc   = "Mark rally start/end + a point-ending reason to a CSV (net-separated
racquet sports)",
105:     .. "(rally_number,start_time,end_time,ending_reason,sport,shots_count; decimal
seconds). "
118:-- Net-separated racquet sports share a forced/unforced-error point-stop taxonomy.
128:local HEADER = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
130:-- reason/sport -> id lookups. POPULATED IN activate(), NOT here: VLC scans an
163:&bull; <b>Net-cord that dribbles over and lands in</b>: live and good &mdash; usually a
<b>winner</b> if unreachable, else judge what happens next. On the SERVE it varies by sport:
tennis/table-tennis = <b>let</b> (replay); badminton net-tick that passes over and lands in =
play on (but a serve shuttle CAUGHT on the net = service_fault); pickleball (2026) = play
on.<br>
166:Output: <i>&lt;video&gt;.rallies.csv</i> next to the video. Full guide:
docs/ENDING_REASONS.md in the repo.<br>
175:local w_reason          -- ending_reason dropdown widget
176:local w_sport           -- sport dropdown widget
185:local w_shots           -- "Number of shots" text input (optional shots_count per rally)
189:local rows = {}         -- in-memory rally rows: { n, s, e, reason, sport, [shots], [extra]
}
258:      return dir .. stem .. ".rallies.csv"
```

```
261:    if d2 then return d2 .. "rally_labels.csv" end
266:  return home .. "rally_labels.csv"
271:-- optional shots_count (blank/missing in older CSVs); any columns beyond it are
291:          sport  = parts[5] or "",
```

24. assistant (2026-06-20T10:04:14.751Z)

I have everything confirmed. The rally-annotator writes `<video>.rallies.csv` with header `rally_number,start_time,end_time,ending_reason,sport,shots_count`. The engine's `gt_loader` reads by header name (start_time/end_time), ignoring the rest. The schemas match cleanly. Let me record the findings.

25. user (2026-06-20T10:05:28.664Z)

Structured output provided successfully

26. assistant (2026-06-20T10:05:39.829Z)

I've mapped the complete "TRAIN YOUR OWN MODEL" loop and returned the findings via StructuredOutput. Key takeaways:

- **The loop is real and runs GPU-free today** for plumbing: annotate in VLC ? `<video>.rallies.csv` ? `python -m backend.worker train` (synth trajectories) ? shells out to `training.gen0.harness` ? versioned `weights/<version>/{weights.json,manifest.json}` bundle. Verified 2026-06-20 on a no-GPU DigitalOcean box (suite green, S3/GCS/MinIO round-trips passed).

- **The schemas MATCH with no transform.** rally-annotator writes `rally_number,start_time,end_time,ending_reason,sport,shots_count`; the engine reads by column name and only needs `start_time`/`end_time` (the default synth path literally does `row["start_time"]`/`row["end_time"]`). The only real prerequisite is the MD5/stem join (byte-identical video, matching filename stem), not a CSV gap.

- **Gating is honest:** a real-trajectory model needs a GPU + the native WASB runner (M3.5, in-flight ? `LinuxNativeRunner` doesn't exist yet). The ship-gate hard-blocks commercial ship regardless of F1 (WASB-C3 provenance unresolved; F1 target uncalibrated). Gemini is inference-only per the licensing guardrail; the bundle attests `no_paid_llm_training_labels`.

One note for your USP framing: be careful labeling this "train your OWN model" as shipping-ready ? today's honest status is `[verified]` for the CPU synth plumbing loop and `[design]/in-flight` for real owned weights (M3.5 + ship-gate gates). The bring-your-own-data ? bundle architecture is real; a production-grade owned model is roadmap.